

sinais `<` e `>` 706

sobrecarga de um template de função 837

template, palavra-chave 706

templates 705

templates de classe 705

templates de função 705

tipo default 714

tipos parametrizados 709

typename, palavra-chave 706

■ Exercícios de autorrevisão

22.1 Indique quais das sentenças a seguir são *verdadeiras* e quais são *falsas*. Em caso de sentenças *falsas*, justifique sua resposta.

- a) Os parâmetros de template de uma definição de template de função são usados para especificar os tipos dos argumentos da função, para especificar o tipo de retorno da função e declarar variáveis dentro da função.
- b) As palavras-chave `typename` e `class` usadas com um parâmetro de tipo de template significam especificamente ‘qualquer tipo de classe definido pelo usuário’.
- c) Um template de função pode ser sobrecarregado por outro template de função com o mesmo nome de função.
- d) Os nomes de parâmetros de template entre definições de template precisam ser exclusivos.
- e) Cada definição de função-membro fora de um template de classe precisa começar com um cabeçalho de template.
- f) Uma função `friend` de um template de classe precisa ser uma especialização de template de função.
- g) Se várias especializações de template de classe são geradas a partir de um único template de classe com um único dado-membro `static`, cada uma

das especializações de template de classe compartilha uma única cópia do dado-membro `static` do template de classe.

22.2 Preencha os espaços em cada uma das sentenças:

- a) Templates nos permitem especificar, com um único segmento de código, uma faixa inteira de funções relacionadas, chamadas _____, ou uma faixa inteira de classes relacionadas, chamadas _____.
- b) Todas as definições de template de função começam com a palavra-chave _____, seguida por uma lista de parâmetros de template delimitada por _____.
- c) Todas as funções relacionadas geradas a partir de um template de função têm o mesmo nome, de modo que o compilador usa a resolução de _____ para chamar a função apropriada.
- d) Os templates de classe também são chamados de tipos _____.
- e) O operador _____ é usado com um nome de template de classe para unir cada definição de função-membro com o escopo do template de classe.
- f) Assim como dados-membro `static` de classes não template, os dados-membro `static` das especializações de template de classe também devem ser definidos e, se necessário, inicializados no escopo de _____.

■ Respostas dos exercícios de autorrevisão

22.1 a) Verdadeiro. b) Falso. As palavras-chave `typename` e `class`, nesse contexto, também permitem um parâmetro de tipo de um tipo fundamental. c) Verdadeiro. d) Falso. Os nomes de parâmetros de template entre templates de função não precisam ser exclusivos. e) Verdadeiro. f) Falso. Ela poderia ser uma função não template.

g) Falso. Cada especialização de template de classe terá sua própria cópia do dado-membro `static`.

22.2 a) especializações de template de função, especializações de template de classe. b) `template`, sinais `<` e `>`. c) sobrecarga. d) parametrizados. e) resolução binária de escopo. f) `namespace global`.

■ Exercícios

22.3 *Template de função de Selection Sort.* Escreva uma função `selectionSort` (ver Apêndice F para obter informações sobre essa técnica de classificação). Escreva

um programa controlador que insira, classifique e imprima um array de `int` e um array de `float`.

- 22.4 Intervalo de array de impressão.** Sobrecarregue o template de função `printArray` da Figura 22.1 de modo que ele use dois outros argumentos inteiros, a saber, `int lowSubscript` e `int highSubscript`. Uma chamada a essa função imprimirá apenas a parte designada do array. Valide `lowSubscript` e `highSubscript`; se um deles estiver fora do intervalo ou se `highSubscript` for menor ou igual a `lowSubscript`, a função sobrecarregada `printArray` deverá retornar 0; caso contrário, `printArray` deverá retornar o número de elementos impressos. Depois, modifique `main` para exercitar as duas versões de `printArray` sobre os arrays `a`, `b` e `c` (linhas 22-24 da Figura 22.1). Não se esqueça de testar todas as capacidades das duas versões de `printArray`.
- 22.5 Sobrecarga de template de função.** Sobrecarregue o template de função `printArray` da Figura 22.1 com uma versão não template que imprima um array de strings de caracteres em um formato organizado, em colunas.
- 22.6 Sobrecarga de operador em templates.** Escreva um template simples para a função predicado `isEqualTo` que compare seus dois argumentos do mesmo tipo com o operador de igualdade (`==`), e retorne `true` se eles forem iguais e `false` se eles forem diferentes. Use esse template de função em um programa que chame `isEqualTo` apenas com uma série de tipos fundamentais. Agora, escreva uma versão separada do programa que chama `isEqualTo` com um tipo de classe definido pelo usuário, mas não sobrecarrega o operador de igualdade. O que acontece quando você tenta executar esse programa? Agora, sobrecarregue o operador de igualdade (com a função operador) `operator==`. O que acontece quando você tenta executar esse programa?
- 22.7 Template de classe Array.** Use um parâmetro não tipo de template `int númeroDeElementos` e um parâmetro de tipo `tipoElemento` para ajudar a criar um template para a classe `Array` (figuras 19.6 e 19.7) que desenvolvemos no Capítulo 19. Esse template permitirá que objetos `Array` sejam instanciados com um número especificado de elementos de um tipo de elemento especificado no tempo de compilação.
- Escreva um programa com template de classe `Array`. O template pode instanciar um `Array` de qualquer tipo de elemento. Sobreponha o template com uma definição específica para um `Array` de elementos `float` (`class Array<float>`). O driver deverá demonstrar a instanciamento de um `Array` de `int` pelo template, e deverá mostrar que uma tentativa de instanciar um `Array` de `float` usa a definição fornecida na `class Array<float>`.
- 22.8** Faça a distinção entre os termos ‘template de função’ e ‘especialização de template de função’.
- 22.9** O que é mais semelhante a um estêncil: um template de classe ou uma especialização de template de classe? Explique.
- 22.10** Qual é a relação entre templates de função e sobrecarga?
- 22.11** Por que você usaria um template de função no lugar de uma macro?
- 22.12** Que problema de desempenho pode resultar do uso de templates de função e de templates de classe?
- 22.13** O compilador realiza um processo de correspondência para determinar qual especialização de template de função deve chamar quando uma função é chamada. Sob quais circunstâncias uma tentativa de fazer uma correspondência resulta em um erro de compilação?
- 22.14** Por que é apropriado referir-se a um template de classe como um tipo parametrizado?
- 22.15** Explique por que um programa em C++ usaria a instrução
- ```
Array<Funcionario> listaTrabalhador(100);
```
- 22.16** Reexamine a resposta a que você chegou no Exercício 22.15. Explique por que um programa em C++ usaria a instrução
- ```
Array<Funcionario> listaTrabalhador;
```
- 22.17** Explique o uso da seguinte notação em um programa em C++:
- ```
template<typename T> Array<T>::Array (int s)
```
- 22.18** Por que você usaria um parâmetro não tipo com um template de classe para um contêiner como um array ou uma pilha?
- 22.19** Suponha que um template de classe tenha o cabeçalho
- ```
template< typename T > class Ct1
```
- Descreva as relações de friend estabelecidas colocando cada uma das seguintes declarações friend dentro desse template de classe. Os identificadores que começam com ‘f’ são funções, aqueles que começam com ‘C’ são classes, aqueles que começam com ‘Ct’ são templates de classe e T é um parâmetro de tipo de template (ou seja, T pode representar qualquer tipo fundamental ou tipo de classe).
- `friend void f1();`
 - `friend void f2( Ct1< T > & );`
 - `friend void C2::f3();`
 - `friend void Ct3< T >::f4( Ct1< T > & );`
 - `friend da classe C4;`
 - `friend da classe Ct5< T >;`
- 22.20** Suponha que o template de classe `Funcionario` tenha um dado-membro `static` contador. Suponha que três especializações de template de classe sejam instanciadas a partir do template de classe. Quantas cópias do dado-membro `static` existirão? Como o uso de cada uma será restringido (se houver restrição)?